Hierarchical layouts are common and well understood by readers. This gives you the option to emphasize the nested container nature of a hierarchy, as we did with the circle pack layout, or the links between parent and child elements, as with the dendrogram.

5.5 Stack layout

You saw the effects of the stack layout in the last chapter when we created a streamgraph, an example of which is shown in figure 5.19. We began with a simple stacking function and then made it more complex. As I pointed out then, D3 actually implements a stack layout, which formats your data so that it can be easily passed to `d3.svg.area` to draw a stacked graph or streamgraph.

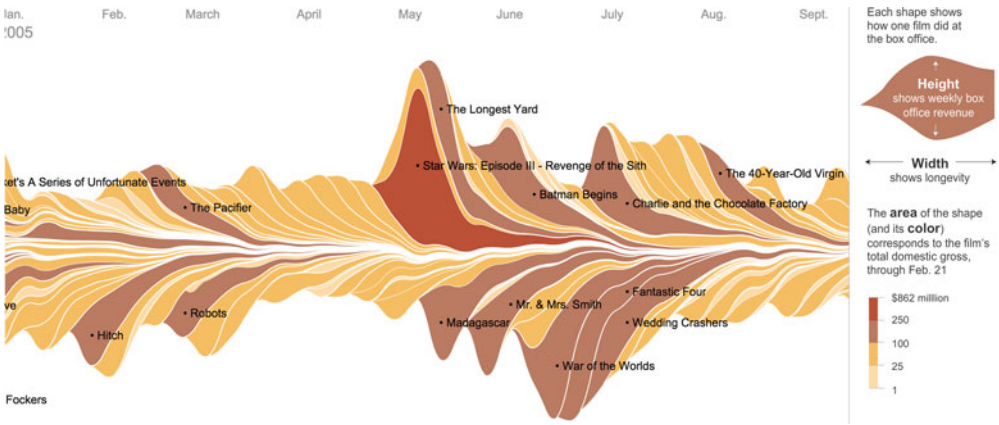


Figure 5.19 The streamgraph used in a *New York Times* piece on movie grosses (figure from *The New York Times*, February 23, 2008; http://mng.bz/rV7M)

To implement this, we'll use the area generator in tandem with the stack layout in listing 5.5. This general pattern should be familiar to you by now:

1. Process the data to match the requirements of the layout.
2. Set the accessor functions of the layout to align it with the dataset.
3. Use the layout to format the data for display.
4. Send the modified data either directly to SVG elements or paired with a generator like `d3.svg.diagonal`, `d3.svg.arc`, or `d3.svg.area`.

The first step is to take our original streamdata.csv data and transform it into an array of movies objects that each have an array of values at points that correspond to the thickness of the section of the streamgraph that they represent.

Listing 5.5 Stack layout example

```
d3.csv("movies.csv", function(error,data) {dataViz(data)});

  function dataViz(incData) {
    expData = incData;
    stackData = [];
```